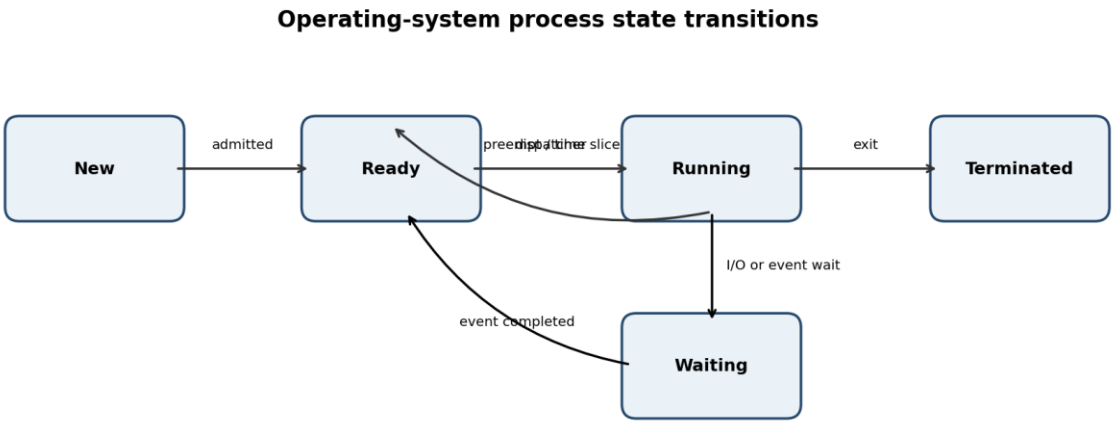
OPERATING SYSTEMS

ASSIGNMENT 2 – CO2

Process Management, Scheduling, Threads, IPC, Synchronisation and Deadlocks



Submitted in partial fulfilment of the Operating Systems course assignment

Student details	Information
Name	S. Maruthu Pandi
Register Number	192511162
Course	CSA0421 – Operating Systems
Assignment	Assignment 2 – CO2
GitHub Repository	https://github.com/Maruthupandi-maker/CSA0421-Operating-System-tree/main/CSA0421%20192511162%20Assignment%202

Q2 – CPU SCHEDULING

2.1 Given job set and formulas

The assignment provides four video-encoding jobs. Arrival time is the time at which a job enters the ready queue, and burst time is the total CPU time required.

Process	Arrival time (AT)	Burst time (BT)	Priority used for comparison*
P1	0	8	3
P2	1	4	1
P3	2	9	4
P4	3	5	2

Priority assumption: The question does not provide numerical priorities. To make the priority comparison calculable, this report uses an explicit illustrative assumption: smaller number means higher priority (P2=1, P4=2, P1=3, P3=4). Replace these values if the instructor supplies a different priority set.

Formulas: Turnaround Time (TAT) = Completion Time (CT) − Arrival Time (AT). Waiting Time (WT) = TAT − Burst Time (BT). Response Time (RT) = First CPU Start Time − Arrival Time. Average value = sum of process values / number of processes.

2.2 First-Come, First-Served (FCFS)

FCFS is non-preemptive. The process that arrives first receives the CPU first and runs until it finishes or blocks. For the supplied arrival order, the schedule is P1 → P2 → P3 → P4.

Gantt chart: 0 | P1 | 8 | P2 | 12 | P3 | 21 | P4 | 26

Process	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	8	8	0	0
P2	12	11	7	7
P3	21	19	10	10
P4	26	23	18	18
Average		15.25	8.75	8.75

Evaluation: FCFS is simple and has low scheduling complexity, but a short job may wait behind a long job. In this data set P4 waits 18 units before its first execution, which is poor for thumbnail requests.

2.3 Shortest Job First (SJF – non-preemptive)

SJF selects the ready process with the smallest CPU burst and lets it run to completion. At time 0 only P1 has arrived, so P1 runs first. At time 8, P2, P3 and P4 are ready; the order is P2 (4), P4 (5), then P3 (9).

Gantt chart: 0 | P1 | 8 | P2 | 12 | P4 | 17 | P3 | 26

Process	CT	TAT	WT	RT
P1	8	8	0	0
P2	12	11	7	7
P3	26	24	15	15
P4	17	14	9	9
Average		14.25	7.75	7.75

Evaluation: SJF gives a lower average waiting time than FCFS for this set. However, it requires knowledge or estimation of CPU burst length. Repeated arrival of short jobs can make a long film wait for a very long time (starvation).

2.4 Shortest Remaining Time First (SRTF – preemptive)

SRTF is the preemptive form of SJF. The scheduler always selects the arrived process with the least remaining CPU time. At time 1, P2 arrives with burst 4, which is shorter than P1's remaining 7, so P1 is pre-empted. At time 3, P4 arrives, but P2 has only 2 units remaining and continues until time 5. Then P4 runs, followed by P1 and P3.

Gantt chart: 0–1 P1 | 1–5 P2 | 5–10 P4 | 10–17 P1 | 17–26 P3

Process	CT	TAT	WT	RT
P1	17	17	9	0
P2	5	4	0	0
P3	26	24	15	15
P4	10	7	2	2

Average		13.00	6.50	4.25

Evaluation: SRTF produces the lowest averages for the supplied set. However, it requires remaining-time estimates and can pre-empt long jobs repeatedly. Starvation is possible when short jobs keep arriving. Frequent pre-emption also adds context-switch overhead.

2.5 Priority scheduling (non-preemptive assumption)

With the stated illustrative priorities (1 is highest), P1 starts at time 0 because it is the only ready process. When P1 completes at time 8, the ready jobs are ordered P2 (priority 1), P4 (2), P3 (4). This gives the same order as the SJF schedule for this particular assumed priority set.

Gantt chart: 0 | P1 | 8 | P2 | 12 | P4 | 17 | P3 | 26

Process	**CT**	**TAT**	**WT**	**RT**
P1	8	8	0	0
P2	12	11	7	7
P3	26	24	15	15
P4	17	14	9	9
Average		14.25	7.75	7.75

Priority scheduling can represent service importance, such as urgent customer jobs or high-priority real-time tasks. A low-priority job may starve. Ageing addresses starvation by gradually increasing the priority of a process the longer it waits.

2.6 Round Robin (RR), quantum = 3

Round Robin gives each ready process a time slice (quantum) of at most 3 time units. If it does not finish, it goes to the back of the ready queue. New arrivals are added to the queue as they arrive.

Gantt chart: 0–3 P1 | 3–6 P2 | 6–9 P3 | 9–12 P4 | 12–15 P1 | 15–16 P2 | 16–19 P3 | 19–21 P4 | 21–23 P1 | 23–26 P3

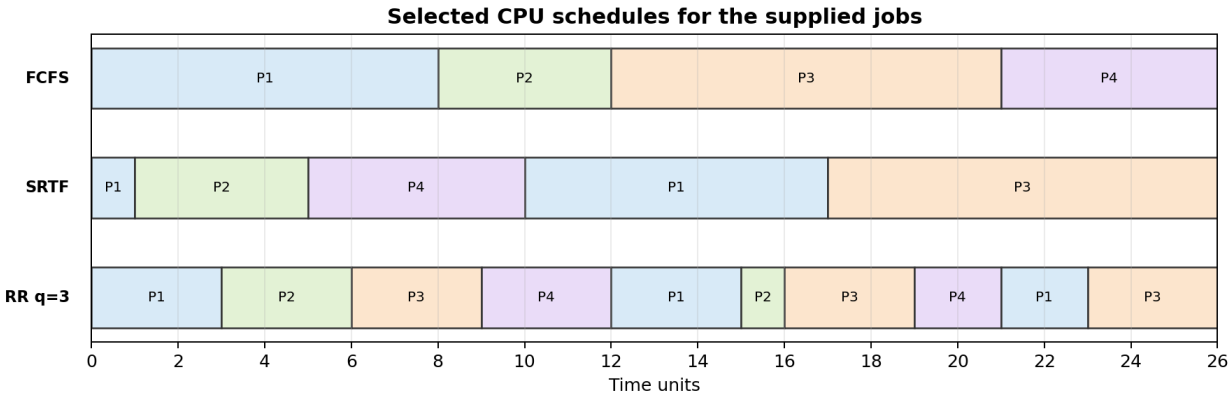


Figure 3. Selected Gantt charts for FCFS, SRTF and Round Robin (quantum 3).

Process	CT	TAT	WT	RT
P1	23	23	15	0
P2	16	15	11	2
P3	26	24	15	4
P4	21	18	13	6
Average		20.00	13.50	3.00

These values match the assignment's expected output. For example, P2 completes at 16, therefore TAT = 16 − 1 = 15 and WT = 15 − 4 = 11.

2.7 Completed Round Robin C program

```
#include <stdio.h>
#define N 4

int main(void) {
    const char *name[N] = {"P1", "P2", "P3", "P4"};
    int arrival[N] = {0, 1, 2, 3};
    int burst[N] = {8, 4, 9, 5};
    int quantum = 3;
    int left[N], finish[N], queue[128], head = 0, tail = 0;
    int added[N] = {0}, time = 0, done = 0;
    for (int i = 0; i < N; ++i) left[i] = burst[i];
    queue[tail++] = 0; added[0] = 1;
    printf("Gantt:");
    while (done < N) {
        if (head == tail) {
            for (int i = 0; i < N; ++i)
                if (!added[i] && arrival[i] > time &&
                    (head == tail || arrival[i] < arrival[queue[head]]))
                    time = arrival[i];
            for (int i = 0; i < N; ++i)
                if (!added[i] && arrival[i] <= time) {
                    queue[tail++] = i; added[i] = 1;
                }
        }
        int p = queue[head++];
        int run = left[p] < quantum ? left[p] : quantum;
        int start = time;
        time += run; left[p] -= run;
        printf(" %s(%d-%d)", name[p], start, time);
        for (int i = 0; i < N; ++i)
            if (!added[i] && arrival[i] <= time) {
                queue[tail++] = i; added[i] = 1;
```

```
            }
        if (left[p] > 0) queue[tail++] = p;
        else { finish[p] = time; ++done; }
    }
    double tw = 0, tt = 0;
    printf("\nProcess TAT WT\n");
    for (int i = 0; i < N; ++i) {
        int tat = finish[i] - arrival[i];
        int wt = tat - burst[i];
        tt += tat; tw += wt;
        printf("%-7s %4d %3d\n", name[i], tat, wt);
    }
    printf("Average TAT = %.2f, average WT = %.2f\n", tt/N, tw/N);
    return 0;
}
```

Compile and execute:

```
gcc -std=c11 -Wall -Wextra rr.c -o rr
./rr
```

Expected result: average turnaround time = 20.00 and average waiting time = 13.50 for quantum 3.

2.8 Effect of Round Robin quantum

Quantum	Number of CPU slices	Process-to-process switches*	Average WT	Average TAT	Interpretation
1	26	23	12.50	19.00	Very responsive time sharing but many switches.
3	10	9	13.50	20.00	Balanced time slice for this workload.

20	4	3	8.75	15.25	Behaves like FCFS because each job finishes within one slice.

*Switches count only a change from one process ID to a different process ID between adjacent slices; they do not count a same-process re-dispatch or an idle-to-process transition. Counts can differ if a measurement tool uses a different definition. Quantum 20 reproduces the FCFS order for this job set.

A small quantum improves response time and fairness but increases the number of scheduling decisions and context switches. A large quantum reduces switching overhead but makes interactive jobs wait longer. The best quantum is a compromise based on typical job length and the operating system's timer and context-switch costs.

2.9 Starvation, ageing and multilevel feedback queue

- Starvation: a process remains ready for a long time because higher-priority or shorter jobs are repeatedly selected.
- Ageing: increase a waiting process's priority after defined intervals. This eventually allows long-waiting jobs to run.
- Multilevel Feedback Queue (MLFQ): newly arrived/interactive jobs begin in a high-priority queue with a short quantum. A job that repeatedly uses its whole quantum is moved to a lower queue with a longer quantum. Periodic priority boosts prevent indefinite starvation.

Recommendation: an MLFQ or carefully tuned Round Robin policy is appropriate for the mixed cloud video service. It gives short thumbnail jobs prompt service while allowing long encoding jobs to use larger CPU slices. The final policy should be validated with the service's real workload distribution; SRTF gives the best calculated average for this small known job set but depends on accurate burst estimates and can starve long films.

2.10 Alternative: MLFQ complexity comparison

With multiple ready queues, a scheduler can select the highest-priority non-empty queue and remove the next job from that queue. With a suitable queue data structure, this selection is typically O(1), whereas a simple SJF implementation that scans all n ready jobs is O(n) per

decision. MLFQ’s benefit is responsiveness without knowing the exact burst in advance. Its risks are tuning complexity and starvation if priority boosts are not used.